

4. LINKED-LIST DATA STRUCTURES

Linked-lists: singly-linked-lists, doubly-linked-lists

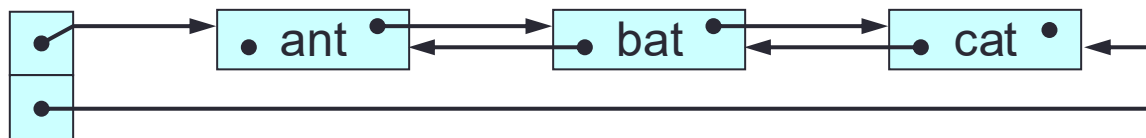
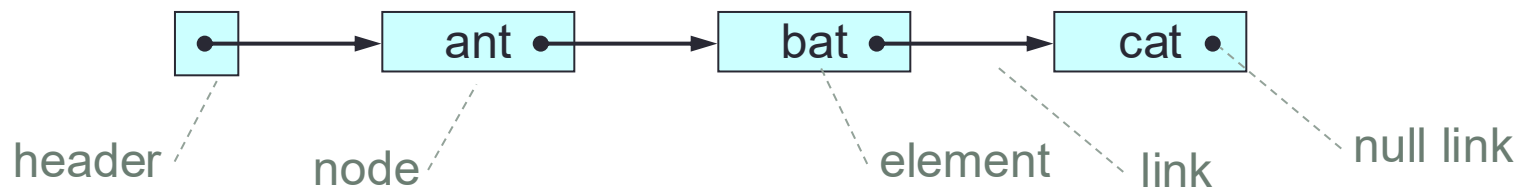
Insertion

Deletion

Searching

Linked-lists (1)

- A **linked-list** consists of a **header** together with a sequence of **nodes** connected by **links**:
 - Each node (except the last) has a **next** node.
 - Each node (except the first) has a **predecessor** node.
 - Each node contains a single element (value or object), plus links to its next and/or predecessor.



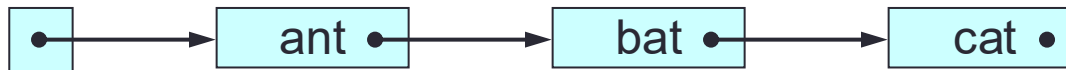
Linked-lists (2)

- The **length** of a linked-list is its number of nodes.
- An **empty** linked-list has no nodes.
- We can manipulate a linked-list's elements.
- We can also manipulate a linked-list's links, and thus change its very structure!

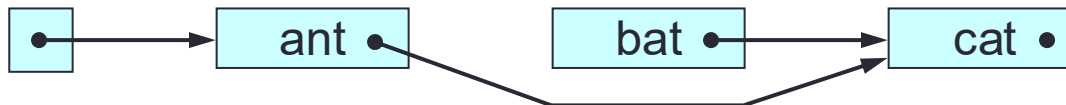
E.g.:

whereas an
array's structure
can't be changed

Before:

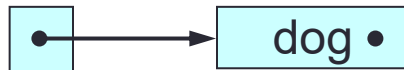
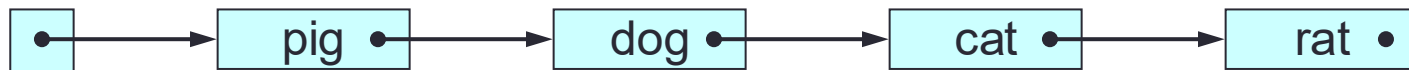


After:



Singly-linked-lists (1)

- A **singly-linked-list (SLL)** consists of a header together with a sequence of nodes connected by links in one direction only:
 - Each SLL node contains a single element, plus a link to the node's **next** node (or null if the node has no **next** node).
 - The SLL header contains a link to the SLL's first node (or a null link if the SLL is empty).



Singly-linked-lists (2) – *Java class*

```
public class SLL<E> {  
    // Each SLL object is the header of a  
    // singly-linked-list.  
    private Node<E> first;  
    public SLL () {  
        // Construct an empty SLL.  
        first = null;  
    }  
    ...  
}
```

Using generics
from the start!

----- methods (to follow)

Singly-linked-lists (3) – Java class contd

```
////////// Inner class //////////  
private static class Node<E> {  
  // Each Node object is a node of a  
  // singly-linked-list.  
  protected E element;  
  protected Node<E> next;  
  public Node (E elem, Node<E> n) {  
    element = elem;  
    next = n;  
  }  
}
```

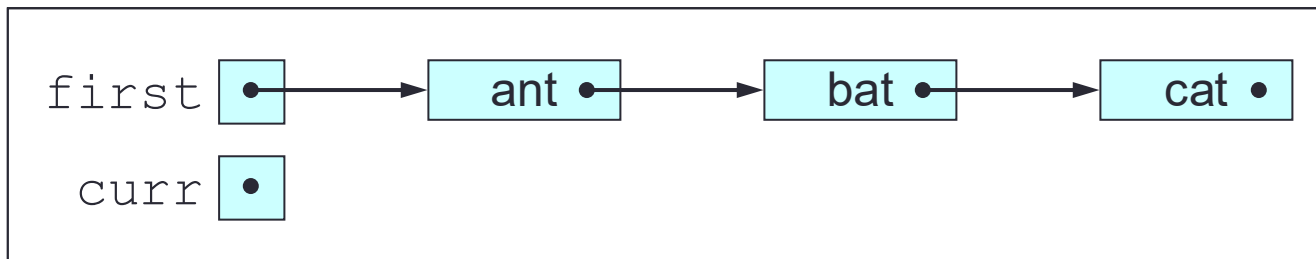
Actually private will
do

Example: SLL traversal

Method to traverse a SLL (in class SLL):

```
public void printFirstToLast () {  
    // Print all elements in this SLL, in first-to-last order.  
    Node<E> curr = first;  
    while (curr != null) {  
        System.out.println(curr.element);  
        curr = curr.next;  
    }  
}
```

Animation:

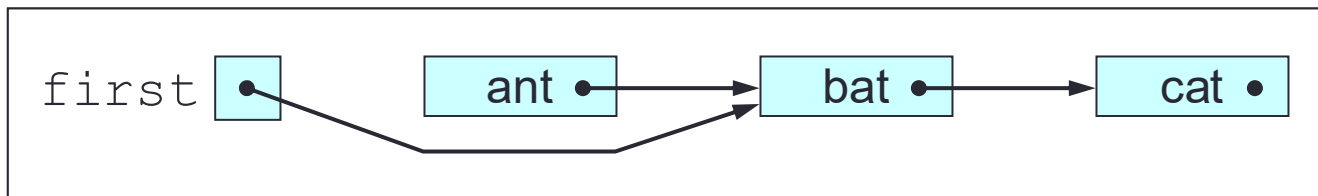


Example: SLL manipulation

- Method to delete a SLL's first node (in class SLL):

```
public void deleteFirst () {  
    // Delete this SLL's first node (assuming length > 0).  
    first = first.next;  
}
```

- Animation:



Insertion

- **Problem:** Insert a new element at a given point in a linked-list.
- Four cases to consider:
 - A. insertion in an empty linked-list;
 - B. insertion before the first node of a non-empty linked-list;
 - C. insertion after the last node of a non-empty linked-list;
 - D. insertion between nodes of a non-empty linked-list.
- The insertion algorithm needs links to the new node's successor and predecessor.

SLL insertion (1)

- **SLL insertion algorithm:**

To insert *elem* at a given point in the SLL headed by *first*:

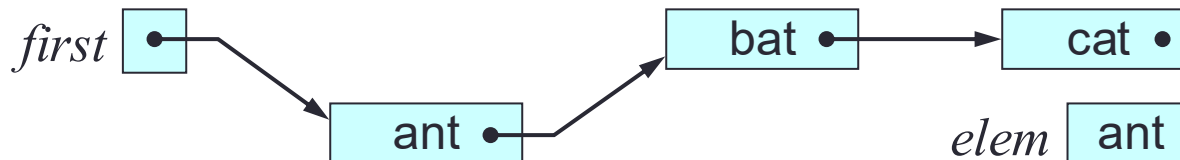
1. Make *ins* a link to a newly-created node with element *elem* and next node null.
2. If the insertion point is before the first node:
 - 2.1. Set *ins*'s next node to *first*. ----- cases A, B
 - 2.2. Set *first* to *ins*.
3. Else, if the insertion point is after the node *pred*:
 - 3.1. Set *ins*'s next node to *pred*'s next node. ---cases C, D
 - 3.2. Set *pred*'s next node to *ins*.
4. Terminate.

SLL insertion (2)

- Animation (insertion before first node):

To insert *elem* at a given point in the SLL headed by *first*:

1. Make *ins* a link to a newly-created node with element *elem* and next node null.
2. If the insertion point is before the first node:
 - 2.1. Set *ins*'s next to *first*.
 - 2.2. Set *first* to *ins*.
3. Else, if the insertion point is after the node *pred*:
 - 3.1. Set *ins*'s next to *pred*'s next.
 - 3.2. Set *pred*'s next to *ins*.
4. **Terminate.**

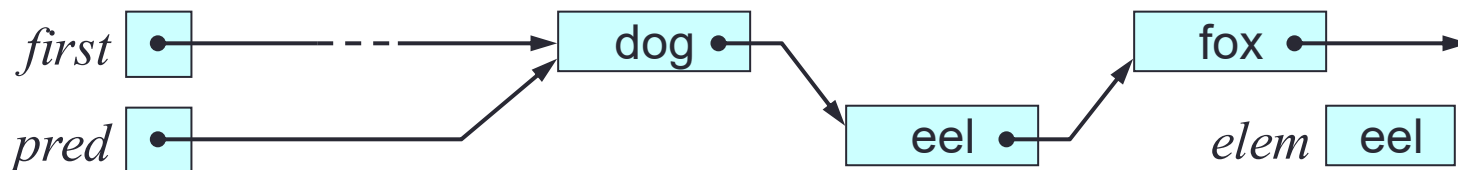


SLL insertion (3)

- Animation (insertion after intermediate node):

To insert *elem* at a given point in the SLL headed by *first*:

1. Make *ins* a link to a newly-created node with element *elem* and next null.
2. If the insertion point is before the first node:
 - 2.1. Set *ins*'s next to *first*.
 - 2.2. Set *first* to *ins*.
3. Else, if the insertion point is after the node *pred*:
 - 3.1. Set *ins*'s next to *pred*'s next.
 - 3.2. Set *pred*'s next to *ins*.
4. **Terminate.**



SLL insertion (4) – *as Java method (in SLL)*

```
public void insert (E elem, Node<E> pred) {  
    // Insert elem at a given point in this SLL, either after the  
    // node pred, or before the first node if pred is null.  
    Node<E> ins = new Node(elem, null);  
    if (pred == null) {  
        ins.next = first;  
        first = ins;  
    }  
    else {  
        ins.next = pred.next;  
        pred.next = ins;  
    }  
}
```

Deletion

- **Problem:** Delete a given node from a linked-list.
- Four cases to consider:
 - A. deletion of a singleton node;
 - B. deletion of the first (but not last) node;
 - C. deletion of the last (but not first) node;
 - D. deletion of an intermediate node.
- The deletion algorithm needs links to the deleted node's successor and predecessor.

SLL deletion (1)

- **SLL deletion algorithm:**

To delete node *del* from the SLL headed by *first*:

1. Let *next* be *del*'s next node.
2. If *del* = *first*:
 - 2.1. Set *first* to *next*. ----- cases A, B
3. Else:
 - 3.1. Let *pred* be *del*'s predecessor. ----- cases C, D
 - 3.2. Set *pred*'s next to *next*.
4. Terminate.

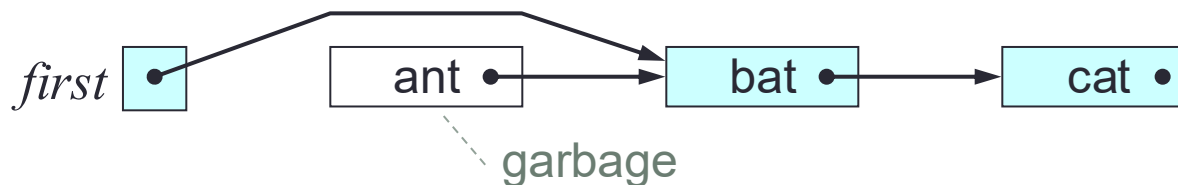
- But there is no link from node *del* to its predecessor, so step 3.1 can access *del*'s predecessor only by following links from *first*!

SLL deletion (2)

- Animation (deleting the first node):

To delete node *del* from the SLL headed by *first*:

1. Let *next* be *del*'s next node.
2. If *del* = *first*:
 - 2.1. Set *first* to *next*.
3. Else:
 - 3.1. Let *pred* be *del*'s predecessor.
 - 3.2. Set *pred*'s next to *next*.
4. **Terminate.**

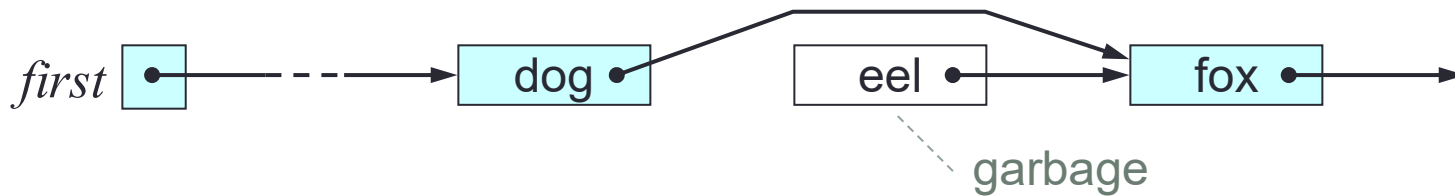


SLL deletion (3)

- Animation (deleting an intermediate node):

To delete node *del* from the SLL headed by *first*:

1. Let *next* be *del*'s next.
2. If *del* = *first*:
 - 2.1. Set *first* to *next*.
3. Else:
 - 3.1. Let *pred* be *del*'s predecessor.
 - 3.2. Set *pred*'s next to *next*.
4. **Terminate.**



SLL deletion (4)

- Analysis:

Let n be the SLL's length.

Step 3.1 must visit all nodes from the first node to the deleted node's predecessor. There are between 0 and $n-1$ such nodes.

Average no. of nodes visited = $(n - 1)/2$

Time complexity is $O(n)$.

SLL deletion (5) – *Java method (in SLL)*

```
public void delete (Node<E> del) {  
    // Delete node del from this SLL.  
    Node<E> next = del.next;  
    if (del == first) first = next;  
    else {  
        Node<E> pred = first;  
        while (pred.next != del)  
            pred = pred.next;  
        pred.next = next;  
    }  
}
```

Searching

- **Problem:** Search for a given target value in a linked-list.
- **Idea:** Follow links from the first node to the last node, terminating when we find a node whose element matches the target value.

SLL searching (1)

- **Unsorted SLL linear search algorithm:**

To find which (if any) node of the SLL headed by *first* contains an element equal to *target*:

1. For each node *curr* in the SLL headed by *first*, repeat:
 - 1.1. If *target* is equal to *curr*'s element, terminate with answer *curr*.
2. Terminate yielding *none*.

SLL searching (2)

- Analysis (counting comparisons):

Let n be the SLL's length.

- If the search is **successful**:

$$\begin{aligned}\text{Average no. of comparisons} &= (n + 1)/2 \\ &\approx n/2\end{aligned}$$

- If the search is **unsuccessful**:

$$\text{No. of comparisons} = n$$

- In either case, time complexity is $O(n)$.

SLL searching (3) *Java method (in SLL)*

- See Moodle (for this and all the previous methods)

Other SLL algorithms

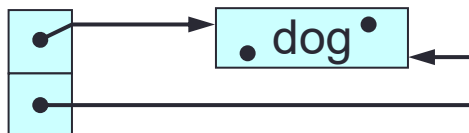
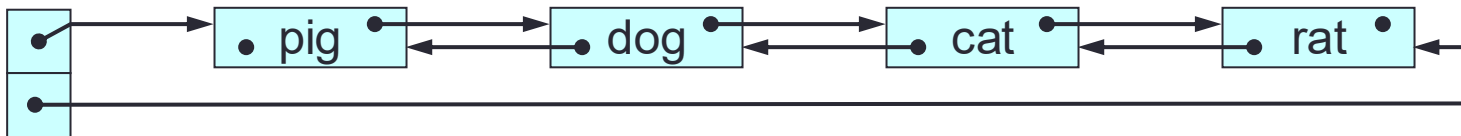
- Some other array algorithms can easily be adapted to SLLs:
 - merging
 - selection sort
 - merge-sort
- But not:
 - binary search.

Implementation of generic SLL can be found on Moodle, together with a class “Animals”, to test the methods.

Also DLL class, coming up..

Doubly-linked-lists (1)

- A **doubly-linked-list (DLL)** consists of a header together with a sequence of nodes connected by links in both direction:
 - Each DLL node contains a single element, plus a link to the node's successor (next) (or null), plus a link to the node's predecessor (or null).
 - The DLL header contains links to the DLL's first and last nodes (or null if the DLL is empty).



Doubly-linked-lists (2) *Java class*

```
public class DLL<E> {  
    // Each DLL object is the header of a  
    // doubly-linked-list.  
    private Node<E> first, last;  
  
    public DLL () {  
        // Construct an empty DLL.  
        first = null;  
        last = null;  
    }  
    ...           ----- methods (to follow)
```

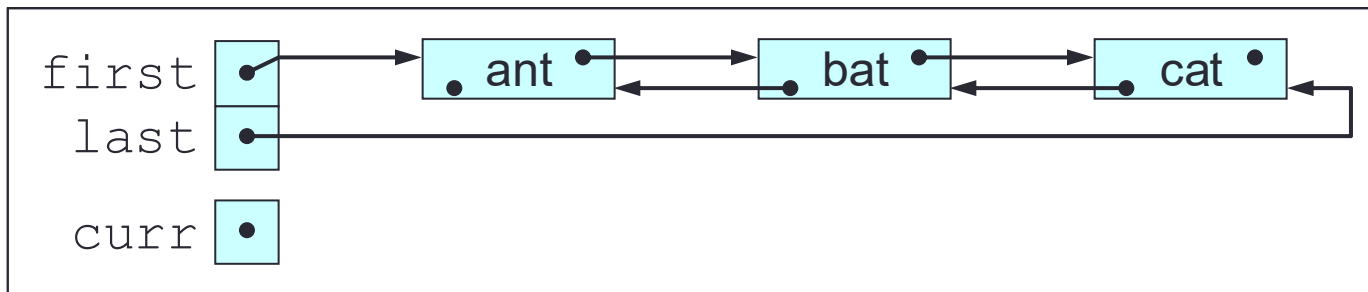
Doubly-linked-lists (3) *Java class contd.*

```
////////// Inner class //////////  
private static class Node<E> {  
    // Each Node object is a node of a  
    // doubly-linked-list.  
  
    private E element;  
    private Node<E> pred, next;  
  
    public Node (E elem, Node<E> pred1, Node<E> next1) {  
        element = elem;  
        pred = pred1; next = next1;  
    }  
}
```

Example: DLL traversal – Java (in DLL)

```
public void printLastToFirst () {  
    // Print all elements in this DLL, in last-to-first order.  
    Node<E> curr = last;  
    while (curr != null) {  
        System.out.println(curr.element);  
        curr = curr.pred;  
    }  
}
```

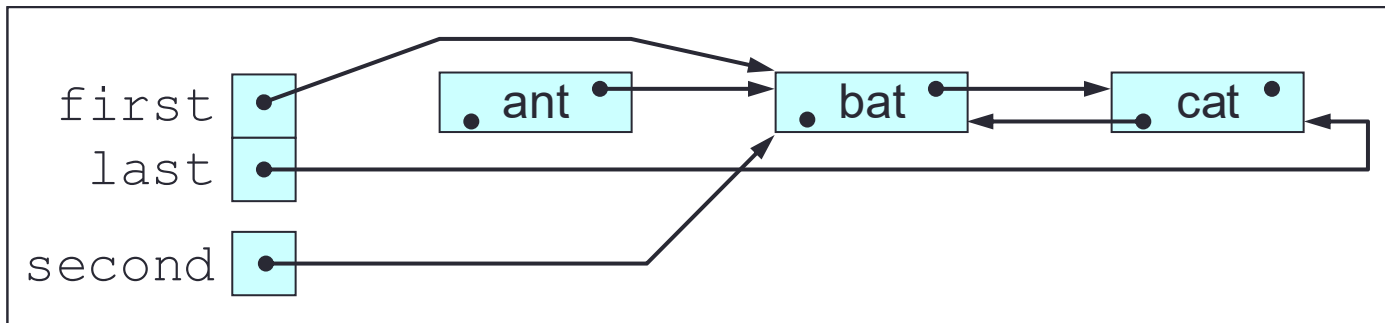
- Animation:



DLL – Java method to delete first node (in DLL)

```
public void deleteFirst () {  
    // Delete this DLL's first node (assuming length > 0).  
    Node<E> second = first.next;  
    second.pred = null;  
    first = second;  
}
```

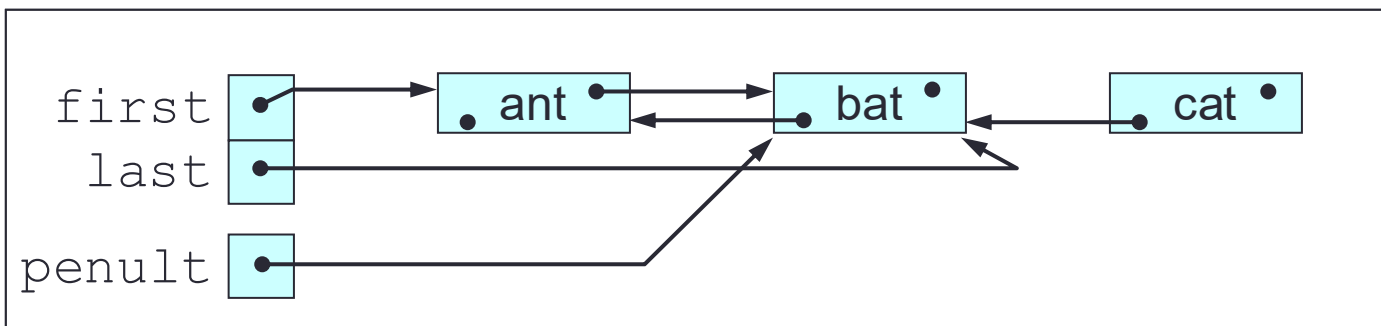
- Animation:



DLL method to delete last node (in DLL)

```
public void deleteLast () {  
    // Delete this DLL's last node (assuming length > 0).  
    Node<E> penult = last.pred;  
    penult.next = null;  
    last = penult;  
}
```

- Animation:



DLL insertion (1)

- **DLL insertion algorithm:**

To insert *elem* at a given point in the DLL headed by (*first*, *last*):

1. Make *ins* a link to a newly-created node with element *elem* and with predecessor and next both null.
2. If *first* = *last* = null:
 - 2.1. Set *first* to *ins*.
 - 2.2. Set *last* to *ins*.
3. Else, if the insertion point is before node *first*:
 - 3.1. Set *ins*'s next to *first*.
 - 3.2. Set *first*'s predecessor to *ins*.
 - 3.3. Set *first* to *ins*.
4. Else, if the insertion point is after node *last*:
 - 4.1. Set *ins*'s predecessor to *last*.
 - 4.2. Set *last*'s next to *ins*.
 - 4.3. Set *last* to *ins*.

DLL insertion (2)

- DLL insertion algorithm (*continued*):
 5. Else, if the insertion point is after the node *pred* and before the node *next*:
 - 5.1. Set *ins*'s predecessor to *pred*.
 - 5.2. Set *ins*'s next to *next*.
 - 5.3. Set *pred*'s next to *ins*.
 - 5.4. Set *next*'s predecessor to *ins*.
 6. Terminate.

DLL deletion

- **DLL deletion algorithm:**

To delete node *del* from the DLL headed by (*first*, *last*):

1. Let *next* be *del*'s next,
and let *pred* be *del*'s predecessor.
2. If *del* = *first*:
 - 2.1. Set *first* to *next*.
3. Else:
 - 3.1. Set *pred*'s next to *next*.
4. If *del* = *last*:
 - 4.1. Set *last* to *pred*.
5. Else:
 - 5.1. Set *next*'s predecessor to *pred*.
6. Terminate.

Java code for full
insertion and
deletion omitted –
see Moodle

Comparison of SLL and DLL algorithms

Algorithm	SLL	DLL
Insertion	$O(1)$	$O(1)$
Deletion	$O(n)$	$O(1)$
Search	$O(n)$	$O(n)$

Insert after given node pred

Delete given node ins

Have to run along whole list in both cases